Move your WordPress blog to headless

A practical, SEO-safe guide using WordPress, Astro and Cloudflare Workers

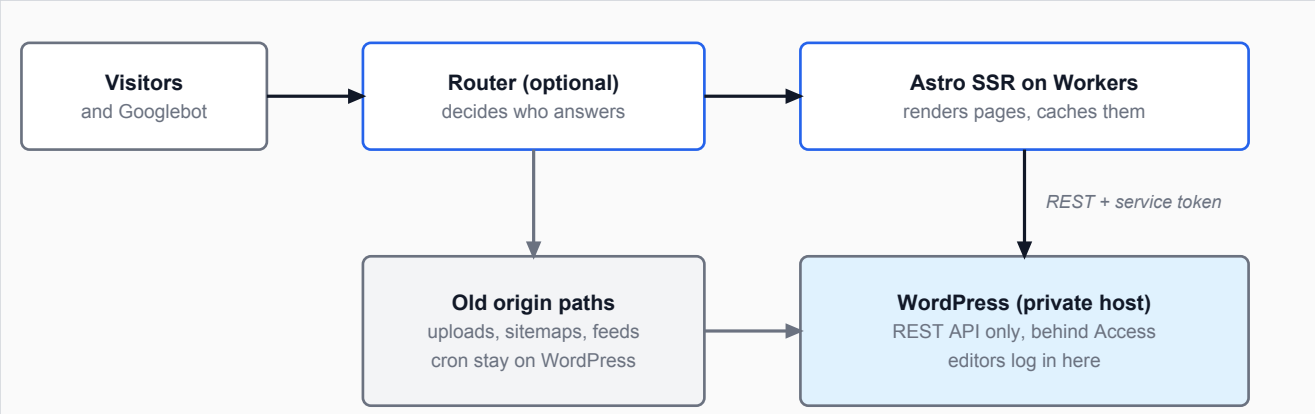
> **What this is.** A field-tested recipe from migrating a real, SEO-heavy WordPress blog (about 180 posts) to a headless setup with **zero changes to its URLs** and a one-minute rollback. You keep WordPress as the editor; visitors get fast pages from a separate front end. Hostnames below are placeholders (`example.com`, `cms.example.com`). Adapt the stack if you prefer another host or framework.

1. Is headless right for you? (be honest first)

Good fit	Poor fit
Content/SEO blogs that want speed and a modern front end, with editors who already love WordPress.	Sites that live on page builders, WooCommerce, membership or heavy form plugins.
Teams that can maintain a small codebase and a deploy pipeline.	Solo bloggers who just need a fast site: a good cache plugin and theme is far cheaper.
Sites where URL stability and Core Web Vitals matter.	Anything relying on theme-rendered shortcodes, widgets and Customizer everywhere.

What you give up: everything the theme used to render. Comments forms, site search, widgets, shortcodes and Customizer settings do not appear on the new pages unless you rebuild them or expose their data over REST. Plan for that list before you start (step 1).

2. The architecture in one picture



Content changes in WordPress ping the Worker, which clears its page cache within about a minute.

WordPress moves to a **private hostname** and is read over its REST API. A Worker renders every page and caches it. An optional router in front keeps the original URL space and sends the things only WordPress can produce (uploads, sitemaps, feeds, WP-Cron) back to the old origin, so nothing about your public URLs changes.

3. The nine steps

Step 1: Inventory and baseline (do not skip)

- List plugins and what each does on the front end (SEO, analytics, author boxes, banners, related posts, comments).

- Note the URL structure: post slugs, categories (nested?), tags, authors, pagination, feeds, sitemaps, trailing slashes.
- **Read your posts-per-page setting.** Ours was 10; a naive build showed 12 and shuffled every paginated URL.
- Save a baseline: crawl your sitemap URLs and store status, title, description, canonical, robots, H1, JSON-LD and body text.

Step 2: Put WordPress on a private host and protect it

Add a hostname such as `cms.example.com` that points at the same server. Protect it with **Cloudflare Access** (people sign in with Google; your front end uses a **service token** with the "Service Auth" action, not "Allow"). WordPress will usually answer on any hostname your web server accepts, but it builds its own URLs from `WP_HOME`, so make the CMS host keep itself:

```
// wp-config.php: only when the request is for the CMS host
if (($_SERVER['HTTP_HOST'] ?? '') === 'cms.example.com') {
    define('WP_HOME',    'https://cms.example.com');
    define('WP_SITEURL', 'https://cms.example.com');
    if (($_SERVER['HTTP_X_FORWARDED_PROTO'] ?? '') === 'https') $_SERVER['HTTPS'] = 'on';
} else {
    define('WP_HOME',    'https://www.example.com/blog/');   // keep your original values
    define('WP_SITEURL', 'https://www.example.com/blog/');
}
```

Step 3: Scaffold Astro on Cloudflare Workers

Use SSR, and read secrets **at request time** with `astro:env`. If you use `import.meta.env`, values are baked into the build output.

```
// astro.config.mjs
import { defineConfig, envField } from 'astro/config';
import cloudflare from '@astrojs/cloudflare';

// read at request time from the Worker's secrets, never baked into the bundle
const secret = (optional = false) =>
  envField.string({ context: 'server', access: 'secret', optional });

export default defineConfig({
  output: 'server',
  adapter: cloudflare({ imageService: 'passthrough' }),
  base: '/blog',                 // only if the blog lives under a path
  trailingSlash: 'always',       // match WordPress
  env: { schema: {
    WORDPRESS_API_URL: secret(),
    CF_ACCESS_CLIENT_ID: secret(true),
    CF_ACCESS_CLIENT_SECRET: secret(true),
  } },
});
```

Two deploy gotchas: add an `.assetsignore` containing `_worker.js` so your server bundle is not published as a public asset, and set secrets with `wrangler secret bulk`, never in the repo.

Step 4: Fetch content and build the routes

```
// lib/wordpress.js
const cms = (url) => fetch(url, {
  redirect: 'manual',   // an Access login redirect must fail loudly, not render as content
  headers: {
    'CF-Access-Client-Id': CF_ACCESS_CLIENT_ID,
    'CF-Access-Client-Secret': CF_ACCESS_CLIENT_SECRET,
  },
});

export async function getPosts({ page = 1, perPage = 10, categories, tags, author } = {}) {
  const u = new URL(`${API}/posts`);
  u.search = new URLSearchParams({ page, per_page: perPage, _embed: 1,
    ...(categories && { categories }), ...(tags && { tags }), ...(author && { author }) });
```

```
  const res = await cms(u);
  if (!res.ok) throw new Error(res.status);
  return { posts: await res.json(), totalPages: +res.headers.get('X-WP-TotalPages') };
}
```

Build routes for: home and `/page/N/`, posts by slug, nested categories (redirect short forms to the canonical path with a 301), tags and authors with pagination, and a real **404** for out-of-range pages (a redirect there is a soft-404).

Step 5: SEO heads straight from the plugin

Do not re-implement SEO. Ask the plugin for the finished <head>. **Rank Math**: enable *Settings, General, Others, Headless CMS Support*, then call `/wp-json/rankmath/v1/getHead?url=...`. **Yoast**: posts already include `yoast_head` in REST.

```
const url = `${API_ROOT}/rankmath/v1/getHead?url=${encodeURIComponent(post.link)}`;
const { head } = await (await cms(url)).json();
// visitors must only ever see public URLs
const html = head.replaceAll('https://cms.example.com', 'https://www.example.com/blog');
// paged archives: reuse page 1's head, then set canonical and og:url to the paged URL
```

Step 6: Keep every URL and behaviour identical

- Same slugs, same trailing slashes, same pagination size, same noindex archives (tags/authors are often noindex).
- Sitemaps, RSS feeds, uploads and WP-Cron **stay on WordPress** (router passes them through). Keep `wp-cron.php` reachable or scheduled posts stop publishing.
- Block or redirect `xmlrpc.php`, public `wp-json` and `wp-admin` on the public host; editors use the CMS host.

Step 7: Fast pages that update within a minute

Cache rendered HTML at the edge for an hour. Make the cache key **build id + a version number in KV + path**: every deploy starts clean, and WordPress bumps the version whenever content changes, so edits appear in about a minute without per-URL purging.

```
// WordPress mu-plugin: ONE ping at the END of the request, after meta and terms are saved
function blog_ping() {
  static $armed = false; if ($armed) return; $armed = true;
  add_action('shutdown', function () {
    wp_remote_post(BLOG_REVALIDATE_URL, [
      'blocking' => false, 'timeout' => 5, 'body' => '{}',
      'headers'  => ['x-revalidate-secret' => BLOG_REVALIDATE_SECRET,
                     'content-type'        => 'application/json'],
    ]);
  });
}
add_action('transition_post_status', function ($new, $old, $post) {
  if ($post->post_type !== 'post') return;
  if ($new === 'publish' || $old === 'publish') blog_ping();
}, 10, 3);
foreach (['created_term', 'edited_term', 'delete_term', 'profile_update'] as $h) {
  add_action($h, 'blog_ping');
}
```

Step 8: Router, staging, then cutover

Path	Goes to
/, /page/N/, /slug/, /category/..., /tag/..., /author/...	Astro (fall back to WordPress if Astro returns 404 for URLs you did not rebuild)
/wp-content/*, sitemaps (*.xml, *.xsl), */feed/, /wp-cron.php, /?s=...	WordPress origin, unchanged
/wp-admin/*, /wp-login.php	Redirect to the CMS host
/xmlrpc.php, /wp-json/*	403

Test the router on a `workers.dev` address first, run your smoke test, and only then attach the route. Keep a `MODE=origin` kill switch that sends everything back to WordPress, and write the rollback down before you need it: **delete the route** and you are back on the old path in about a minute.

Step 9: Validate like a skeptic, then watch

Crawl every URL from your sitemaps (plus archives and every paginated page) on both the old and new site and diff: status, title, description, robots, canonical, H1, JSON-LD, headings, images, **link attributes** and body text. Pace your requests: your own crawler can trip rate limiting (HTTP 429) or a bot challenge. After launch watch Search Console (indexing, crawl errors, canonicals) for two weeks.

4. Gotchas we actually hit

Symptom	Cause	Fix
Paginated pages hold different posts; the last pages 404	Front end used 12 per page; WordPress used 10	Match "posts per page" exactly
Thousands of internal links suddenly **nofollow** and open in a new tab	Rank Math (Nofollow External Links) runs on the CMS host, where your public domain looks external	Normalise link attributes in the front end: own domain plain, external always nofollow
Secrets visible in the build	`import.meta.env` inlines values	Use `astro:env` with `access: "secret"`
Server code reachable as a static file	Adapter output published as assets	Add `.assetsignore` with `_worker.js`
Old pages served after a deploy	Cache survived the release	Put a build id in the cache key
Edits show up, but banner/author box do not	They live in widgets, Customizer or user meta, not in post content	Expose them through a tiny read-only REST route or REST field
Slash-less URLs redirect to a raw server address	WordPress built the redirect from the proxy's host header	Let the front end/router issue the redirect to the public URL
Access login instead of JSON for your front end	Policy action was "Allow" for the service token	Use the "Service Auth" action
Cron jobs stop	wp-cron.php blocked	Pass it through to WordPress

> **Privacy tip.** Old author boxes often print authors' email addresses (sometimes only lightly obfuscated). Do not copy that into the new front end.

5. How long does it take?

Blog size	Realistic effort
Up to ~200 posts, standard plugins	A weekend for a working copy (steps 1 to 5); a few more days for URL parity, caching, router and a crawl-diff; then two weeks of watching.
Many custom plugins/shortcodes	Budget extra time per plugin to rebuild or expose its data. This is where the real cost hides.

6. Launch checklist

- [ ] Baseline crawl saved; posts-per-page, URL structure and plugins documented
- [ ] CMS host private and behind Access; service token works (Service Auth)
- [ ] Secrets set via the platform, not the repo; no secrets in the bundle
- [ ] SEO head from the plugin on every page type; paged canonicals correct
- [ ] Sitemaps, feeds, uploads and WP-Cron pass through to WordPress
- [ ] Link attributes match the old site (internal plain, external nofollow)
- [ ] Edge cache with build id + version; WordPress ping tested with a real edit
- [ ] Staging router tested; smoke test written; **rollback steps written and rehearsed**
- [ ] Full crawl-diff shows only intentional differences
- [ ] Search Console watched for two weeks

Credit and further reading: the idea of a headless WordPress front end on Cloudflare was popularised in a freeCodeCamp tutorial on building one with Astro SSR on Cloudflare Pages. See also the official Astro, Cloudflare Workers and Cloudflare Access documentation, and your SEO plugin's headless docs (Rank Math or Yoast).